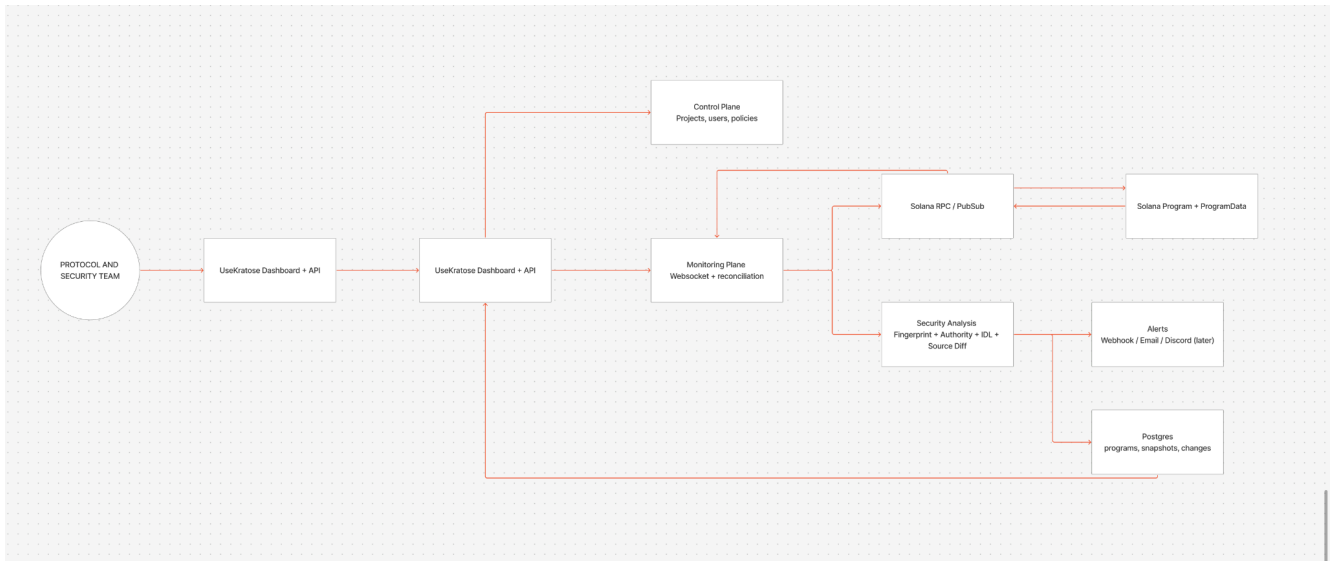


USEKRATOSE

Continuous Security Verification for Solana Programs

Master Product Documentation • Technical Architecture • MVP Specification •
Roadmap • Go-to-Market



Prepared for the Colosseum Global Hackathon • September 2026

Document Status & Working Assumptions

This document is the master product and engineering specification for UseKratose. It is intentionally broader than the hackathon MVP so that the team can distinguish what must be shipped now from what belongs in later phases. It should be treated as a living source of truth: changes to scope, architecture, product positioning, or commercial strategy should be reflected here before they are scattered across code, chats, or slide decks.

UseKratose is designed first for Solana. The system observes public on-chain state and publicly available metadata. The MVP does not make guarantees that a program is safe, does not replace a professional security audit, and must never present an automatically generated risk label as definitive proof of security. Its job is to establish verifiable facts about deployments and upgrades, explain relevant changes, and make drift visible.

Item	Decision
Primary network	Solana
Primary category	Security Tools
Core product	Continuous verification of deployed programs after deployment/audit
MVP customer	Solana protocol teams and security engineers
Hackathon goal	Working product + real monitored programs + clear venture story
Authoritative signals	On-chain state, cryptographic fingerprints, signed metadata, deterministic diffs
AI role	Explanation and prioritization; never the sole source of truth

Executive Summary

UseKratose is a continuous security verification layer for Solana programs. Traditional audits are point-in-time assessments. A protocol can be audited, deploy a known binary, then later upgrade its program, change its upgrade authority, alter its IDL, or modify privileged behavior. Users, integrations, wallets, and even the original auditor may continue to rely on the earlier security assumption even though the deployed system has changed.

UseKratose closes that gap by creating a baseline security snapshot for a Solana program, continuously monitoring the relevant on-chain accounts, detecting finalized changes, fingerprinting the deployed executable, inspecting authority and metadata changes, comparing the new state with the previous trusted state, and generating a structured security event. When source/IDL/verified-build information is available, the platform enriches the event with human-readable semantic differences.

The hackathon MVP is deliberately narrow: add a Solana program, resolve its ProgramData account, create a baseline snapshot, detect an upgrade or control-plane change, compute a new fingerprint, produce a deterministic diff, display a security status change, and deliver an alert. Everything else including auditor workflows, wallet integrations, enterprise policy engines, broad automated static analysis, and cross-chain expansion is secondary until the core loop works reliably.

The commercial model is recurring SaaS for protocols and security teams, plus a security-status API for wallets, explorers, aggregators, exchanges, and developer infrastructure. A free public program profile acts as the acquisition layer; paid monitoring, change analysis, webhooks, retention, and integrations create recurring value.

Table of Contents

1. Product Thesis and Problem Definition
2. Why Solana and Why Now
3. Scope, Non-Goals and Product Boundaries
4. User Personas
5. Jobs to Be Done
6. Core User Journeys
7. Product Requirements
8. MVP Definition
9. MVP Screen-by-Screen Specification
10. System Context
11. Solana Program Deployment Model
12. ProgramData Resolution
13. Upgrade Detection
14. Reliability and Reconciliation
15. Snapshot and Fingerprinting Model
16. Upgrade Authority Analysis
17. IDL and Program Metadata
18. Verified Builds Integration
19. Source and Semantic Diffing
20. Security Event Model
21. Risk Classification
22. AI Explanation Layer
23. Backend Architecture
24. Monitoring Worker
25. Frontend Architecture
26. Database Design
27. API Design
28. Authentication and API Keys
29. Webhooks and Notifications
30. Caching and Rate Limiting
31. Error Handling
32. Observability
33. Security of UseKratose
34. Privacy and Data Handling
35. Testing Strategy
36. Test Fixtures and Demo Program

- 37. Deployment and Environments
- 38. Cost-Minimized Infrastructure
- 39. Performance and Scaling
- 40. Product Analytics
- 41. Pricing and Packaging
- 42. Go-to-Market
- 43. Distribution Loops
- 44. Competitive Positioning
- 45. Sales Playbook
- 46. Customer Discovery
- 47. Colosseum Strategy
- 48. Demo Script
- 49. Four-Week Hackathon Roadmap
- 50. Post-Hackathon Roadmap
- 51. KPI Framework
- 52. Risk Register
- 53. Team Operating Model
- 54. Definition of Done
- 55. Backlog and Future Modules
- 56. Appendices

1. Product Thesis and Problem Definition

The Core Thesis

The security state of an upgradeable on-chain program is not permanent. A conventional audit answers a historical question: what did a specific source tree, build artifact, or deployed version look like at the time it was reviewed? UseKratose answers the ongoing question: what is actually deployed now, how has it changed, and do previously trusted assumptions still apply?

The product therefore treats every meaningful deployment or authority change as a security lifecycle event. It does not wait for an exploit or user report. Instead it maintains a continuously refreshed chain of evidence from program address to ProgramData, executable hash, authority, metadata, IDL, verified-build information, and human-readable change summary.

Problem statement

- A program may be upgraded after an audit or verification event.
- The public may continue to associate the old audit with the new deployment.
- Upgrade authority can change independently of business logic, materially changing control risk.
- Program IDs remain stable while executable contents can change, so address-based trust is insufficient.
- Existing monitoring can tell teams that something changed; UseKratose aims to explain the security significance of that change and preserve deployment history.

The one-sentence product definition

UseKratose continuously verifies deployed Solana programs and detects when upgrades or control changes invalidate previously trusted security assumptions.

The product promise

If a monitored program changes, UseKratose should know quickly, independently confirm the finalized state, preserve an immutable snapshot, explain the difference using deterministic evidence, and notify the people or systems that rely on that program.

2. Why Solana and Why Now

Solana is a strong first network because programs have a clear deployment and upgrade model. Programs deployed through the upgradeable loader have a Program account and a ProgramData account; the latter stores executable bytes and deployment metadata. Solana RPC also exposes account subscriptions that can notify a client when account data changes. These primitives make continuous verification possible without operating a validator. [R1][R2]

Solana also has an expanding ecosystem around verified builds and canonical program metadata. Verified builds connect deployed executables with public source repositories through deterministic builds and hashes, while program metadata can expose canonical IDLs and security information signed by an upgrade authority. UseKratose can compose these existing trust primitives instead of inventing a proprietary source of truth.

[R4][R5]

The timing is favorable for a startup because protocol complexity, institutional adoption, and composability increase the blast radius of silent or poorly understood upgrades. The more integrations a protocol has, the more parties need to know whether the code they analyzed yesterday is still the code running today.

Solana-specific wedge

Solana primitive	UseKratose use
Program ID	Stable identity of the monitored program
ProgramData account	Executable bytes and deployment state for loader-v3 programs
accountSubscribe	Low-cost near-real-time change signal
getAccountInfo	Authoritative HTTP reconciliation
Upgrade authority	Control-plane risk signal
Program metadata / IDL	Semantic context for change analysis
Verified builds	Source-to-deployment authenticity evidence

3. Scope, Non-Goals and Product Boundaries

MVP scope

- Monitor explicitly added Solana programs on devnet and mainnet where RPC access permits.
- Resolve and persist ProgramData for upgradeable loader-v3 programs.
- Create cryptographic deployment snapshots.
- Detect finalized program or authority changes using PubSub plus reconciliation.
- Record version history.
- Compare bytecode hashes, authority state, deployment slot, IDL/metadata hashes, and source/verification information when available.
- Generate structured security events with evidence.
- Display current status, history, and diff in a web dashboard.
- Send at least one reliable alert type, preferably webhook and in-app.

Explicit non-goals for the hackathon

- Full formal verification of arbitrary programs.
- Claiming that a hash match means a program is safe.
- Replacing professional auditors.
- Monitoring every Solana program globally from day one.
- Running our own Solana validator or archival RPC infrastructure.
- Cross-chain support.
- A token, DAO, or protocol governance layer.
- A two-sided auditor marketplace.
- Automated exploit execution against mainnet programs.

Scope discipline is strategic. Colosseum explicitly evaluates whether teams prioritize development well; a focused and reliable core loop is stronger than a broad but shallow security suite. [R6]

4. User Personas

Persona	Primary pain	What they buy
Protocol engineer	Needs to know when deployed program state differs from expected releases	Continuous monitoring, release verification, alerts
Security lead	Needs evidence of upgrade/control changes across multiple programs	Portfolio monitoring, diff history, policy alerts
External auditor	An audit can become stale after client upgrades	Client portfolio monitoring and re-review triggers
Wallet / explorer team	Needs machine-readable trust context before displaying/interacting	Security-status API
Integrator / aggregator	Depends on external programs and needs change visibility	Dependency monitoring, webhooks
Institution / exchange	Needs governance and code-change visibility for integrated protocols	Enterprise policies, retention, reporting

Primary persona for the MVP

The primary persona is a Solana protocol engineer or security-minded founder who can paste a program address, understands what an upgrade means, and values fast evidence. This persona can validate the product without requiring enterprise procurement or consumer education.

5. Jobs to Be Done

Jobs-to-be-done keep the product anchored in outcomes rather than feature accumulation.

- When I deploy or integrate a Solana program, I want a durable baseline so I can prove what was running at a given time.
- When the deployed program changes, I want to know quickly so that I do not continue assuming the previous version is active.
- When an authority changes, I want to understand whether control becomes more centralized or unexpected.
- When a change occurs, I want a concise explanation of exactly what evidence changed so that I can decide whether human review is needed.
- When I am responsible for many programs, I want one place to see which ones are stable, changed, immutable, or require attention.
- When another application depends on a program, I want an API/webhook so that program security state can be integrated into my own workflow.

6. Core User Journeys

Journey A: Add and baseline a program

1. User signs in and creates a project.
2. User pastes a Solana Program ID and selects cluster.
3. UseKratose validates the address and fetches account state.
4. System identifies loader type and, for loader-v3 upgradeable programs, resolves ProgramData.
5. System fingerprints the executable, reads authority and deployment slot, and fetches optional metadata.
6. System stores snapshot #1 and shows monitoring status.

Journey B: Detect an upgrade

7. WebSocket notification or reconciliation indicates ProgramData changed.
8. Worker waits for/reads finalized state.
9. New snapshot is created only if the fingerprint or tracked metadata differs.
10. Diff engine compares snapshots.
11. Security event is classified.
12. Dashboard status changes and alert is delivered.

Journey C: Review a change

13. User opens event.
14. User sees facts first: slots, hashes, authorities, metadata hashes, verification state.
15. If IDL/source context exists, user sees semantic differences.
16. AI explanation summarizes implications and clearly cites the deterministic facts it used.
17. User can mark event reviewed and optionally attach a note.

7. Product Requirements

Functional requirements

ID	Requirement	Priority
FR-001	Add a program address and cluster	P0
FR-002	Resolve program deployment model and ProgramData where applicable	P0
FR-003	Create a baseline snapshot with executable hash	P0
FR-004	Subscribe to relevant account changes	P0
FR-005	Run periodic reconciliation	P0
FR-006	Persist version history	P0
FR-007	Detect upgrade authority changes	P0
FR-008	Generate deterministic diff	P0
FR-009	Show current status and event timeline	P0
FR-010	Send webhook/in-app alert	P0
FR-011	Fetch IDL/program metadata when available	P1
FR-012	Integrate verified-build metadata	P1
FR-013	Produce AI-assisted explanation	P1
FR-014	Project-level API key	P1
FR-015	Public program profile	P2

Non-functional requirements

Area	Target for MVP
Correctness	Never claim an upgrade from an unconfirmed transient read; reconcile finalized state
Availability	Best effort on free infrastructure; clear degraded status
Latency	Detect WebSocket change quickly; finalized event typically within network/provider delay
Idempotency	Duplicate notifications must not create duplicate snapshots/events
Auditability	Every status must link to raw facts and observation time
Security	No private keys required to monitor a public program
Cost	No validator requirement; free/low-cost managed services first

8. MVP Definition

The MVP is not a generalized smart-contract auditor. It is a continuous deployment-security monitor. The product is successful when a judge or external protocol team can watch a real Solana upgrade occur and see UseKratose detect, verify, diff, persist, and explain it without manual intervention.

P0 feature set

- Email or simple OAuth sign-in
- Project creation
- Add program
- Program overview
- Baseline snapshot
- Real-time watcher
- Reconciliation worker
- Version history
- Authority state
- Change event
- Diff view
- Webhook or in-app notification
- Demo program that can be upgraded during presentation

MVP acceptance test

Starting from a clean account, a user can add the hackathon demo program. UseKratose establishes a baseline. The team deploys a second version to devnet using the same Program ID. Without refreshing the hidden admin state, the product detects the finalized change, records a new hash and slot, identifies whether the authority changed, surfaces an event on the dashboard, and produces a clear explanation. Replaying the same notification must not duplicate the event.

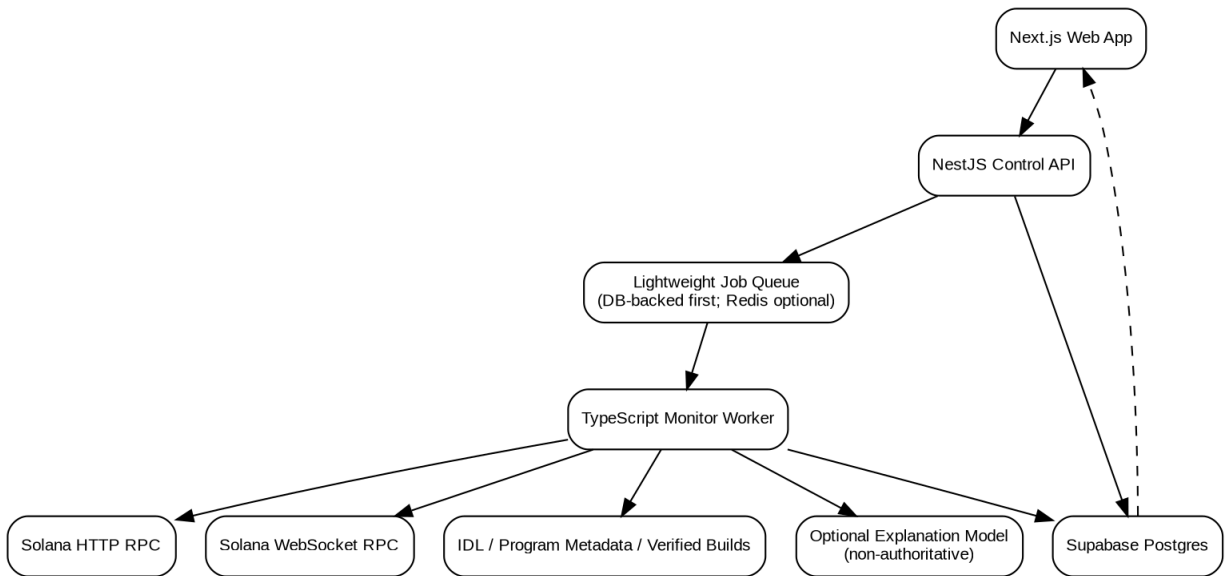


Figure: recommended low-cost MVP architecture.

9. MVP Screen-by-Screen Specification

Screen	Purpose	Key elements
Landing	Explain problem in <10 seconds	Headline, live example, CTA to scan/add program
Sign in	Low-friction access	Email/OAuth
Project dashboard	Portfolio status	Programs, current status, recent changes
Add program	Onboard monitor	Address, cluster, optional name
Program detail	Single source of truth	Program ID, ProgramData, authority, hash, slot, metadata, status
Version history	Evidence chain	Snapshots by slot/time/hash
Change detail	Explain event	Before/after facts, semantic diff, severity, explanation
Settings	Delivery controls	Webhook URL, API keys later
Public profile P2	Acquisition/trust surface	Objective deployment facts only

Status vocabulary

Avoid ambiguous labels such as SAFE/UNSAFE. UseKratose should prefer statuses that describe observable state: Stable, Change Detected, Review Recommended, Verification Stale, Immutable, Monitoring Degraded, Metadata Unavailable. Security severity belongs to an event, not an absolute guarantee about the entire protocol.

10. System Context

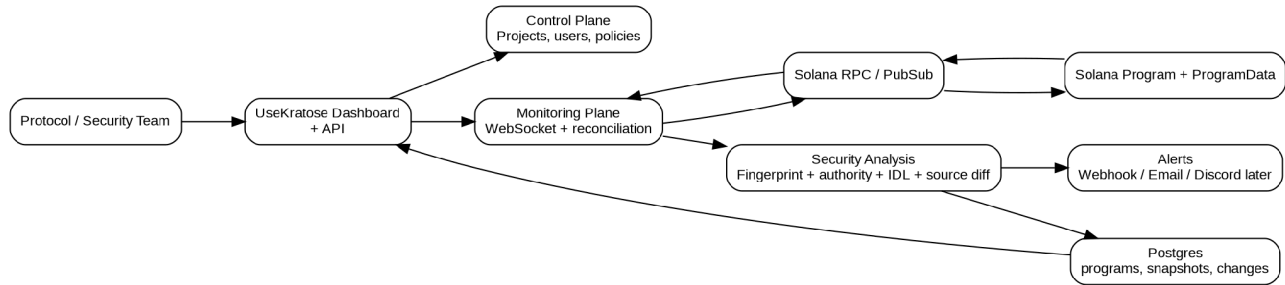


Figure: UseKratose system context.

The architecture separates the user-facing control plane from the monitoring plane. The control plane manages accounts, projects, monitored programs, alert destinations, API keys, and product configuration. The monitoring plane maintains subscriptions, schedules reconciliation, fetches state, creates snapshots, and generates security events. This separation prevents dashboard/API traffic from interfering with long-lived WebSocket workloads.

11. Solana Program Deployment Model

Solana programs are executable accounts containing sBPF bytecode. Programs are stateless in the sense that mutable application state resides in separate accounts. For loader-v3 upgradeable programs, the Program account is the stable Program ID and points to a ProgramData account that stores the deployed executable and upgrade metadata. Removing the upgrade authority makes the program immutable. [R1][R2]

Why ProgramData matters

A naive monitor that only watches the Program ID can miss the most important change because the stable Program account can remain the same while the ProgramData contents change. UseKratose therefore resolves the loader model first and determines the correct account or accounts to watch.

Loader compatibility

The implementation should explicitly inspect the owner/loader rather than assume every executable account uses the same deployment model. The hackathon release can support loader-v3 first, with an extension interface for other loader models. Unsupported loader types should be reported honestly rather than misclassified.

12. ProgramData Resolution

Resolution converts a user-provided Program ID into the deployment objects that UseKratose needs to monitor. The resolver fetches the program account, verifies that it is executable, checks its owner, parses loader state, and extracts the ProgramData address when the loader model supports it.

```
type ResolvedProgram = {  
  programId: string;  
  cluster: "devnet" | "mainnet-beta";  
  loader: "bpf-upgradeable" | "other";  
  programDataAddress?: string;  
  supported: boolean;  
};
```

Resolution invariants

- Never accept a malformed base58 key as a program.
- Do not treat a non-executable account as a program.
- Persist the raw owner and raw account data hash for debugging.
- Re-resolve when loader/owner assumptions no longer match.
- Keep parsing code versioned and covered by fixture tests.

13. Upgrade Detection

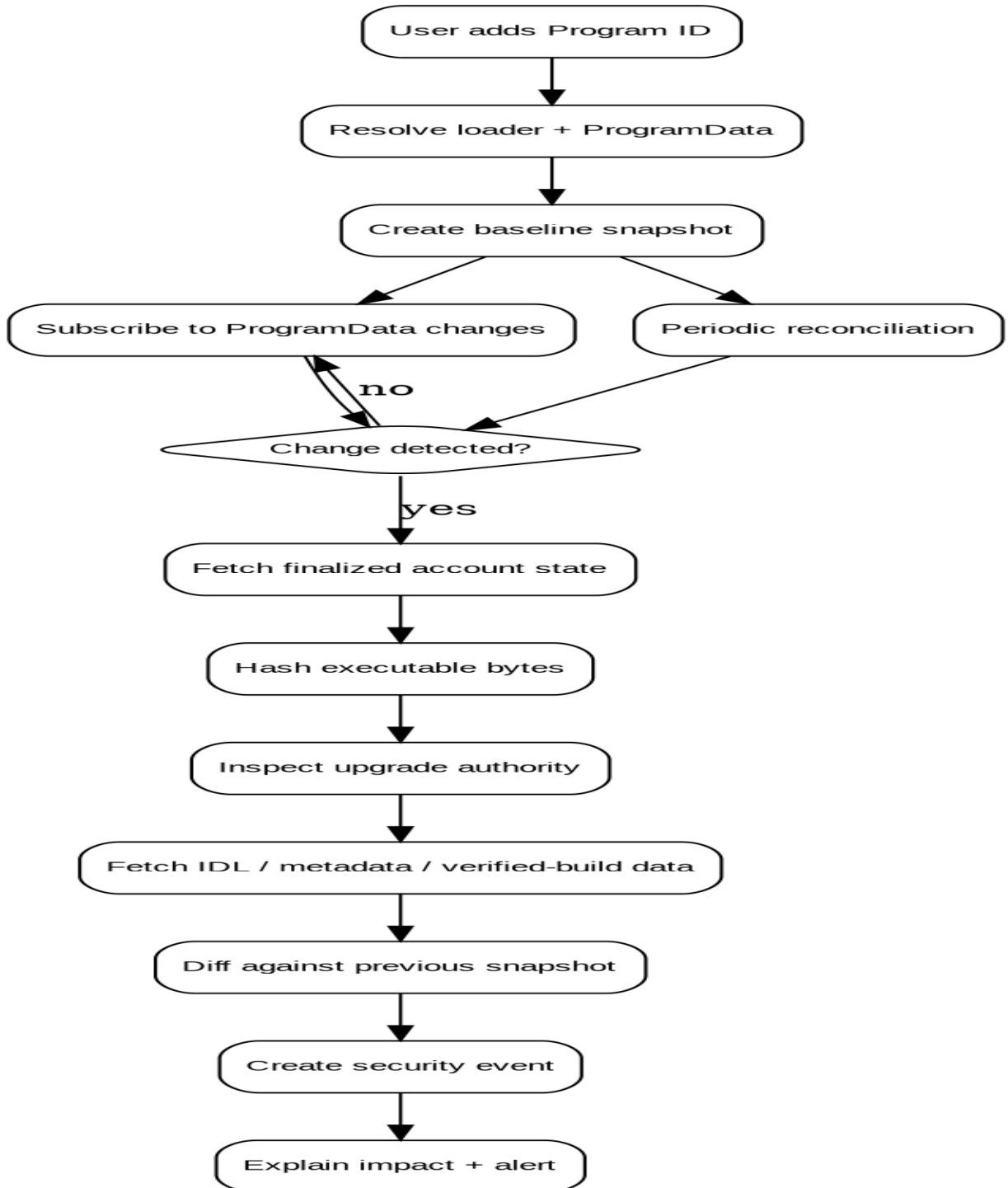


Figure: upgrade/change detection pipeline.

The primary real-time signal is Solana PubSub accountSubscribe on the ProgramData account. The RPC method notifies subscribers when an account's lamports or data changes. UseKratose uses this as a trigger, not as the final authority. A notification schedules a finalized fetch and diff rather than immediately declaring a security event. [R3]

Detection algorithm

```
onAccountNotification(programId, notification):
  enqueue("reconcile-program", {
    programId,
    reason: "pubsub",
    observedSlot: notification.context.slot
  })

reconcile(programId):
  current = fetchFinalizedProgramState(programId)
  previous = latestSnapshot(programId)
  if fingerprint(current) == fingerprint(previous):
    return NO_CHANGE
  next = persistSnapshot(current)
  event = diff(previous, next)
  publish(event)
```

This design naturally deduplicates noisy notifications because the snapshot fingerprint is the idempotency key for meaningful state.

14. Reliability and Reconciliation

WebSocket connections are fast but not sufficient as the sole source of truth. Providers restart, networks flap, subscriptions drop, and the application itself can be unavailable. UseKratose therefore uses a dual-path model: PubSub for speed and periodic HTTP reconciliation for completeness.

Recommended MVP schedule

Mechanism	Cadence	Purpose
accountSubscribe	Continuous	Immediate change trigger
HTTP reconcile active programs	Every 5–15 minutes	Catch missed notifications
Subscription health check	Every 30–60 seconds	Reconnect dropped sockets
Full portfolio sweep	Every 1–6 hours	Correct any drift in scheduler state

Exactly-once effect, at-least-once processing

Workers may process the same change more than once; the database must ensure only one canonical snapshot/event is produced for a given program and fingerprint transition. This is simpler and more reliable than trying to guarantee exactly-once message delivery in a hackathon stack.

15. Snapshot and Fingerprinting Model

A snapshot is the evidence bundle for a specific observed deployment state. The fingerprint should be deterministic: the same finalized on-chain state must produce the same fingerprint regardless of which worker or RPC provider fetched it.

Snapshot fields

Field	Meaning
program_id	Stable program address
programdata_address	Resolved executable data account when applicable
slot	Finalized context/deployment observation slot
executable_hash	SHA-256 or BLAKE3 hash of normalized executable bytes
account_data_hash	Hash of full tracked account payload
upgrade_authority	Current authority or null
idl_hash	Hash of normalized IDL when available
metadata_hash	Hash of canonical metadata when available
verified_build_state	Known verification/source state
observed_at	UseKratose observation timestamp

Normalization

Semantic inputs such as JSON IDLs must be canonicalized before hashing: stable key ordering, normalized whitespace, and consistent encoding. Otherwise formatting-only changes would look like meaningful metadata changes.

16. Upgrade Authority Analysis

Control-plane risk is as important as executable code. A program can remain byte-for-byte identical while its upgrade authority moves from a multisig-controlled address to a single key, or from one organization to another. UseKratose records every authority transition as a first-class event.

MVP authority facts

- Current upgrade authority address or immutable/null state.
- Previous authority address.
- Whether the authority changed.
- Whether the program became immutable.
- Whether an immutable program unexpectedly appears mutable under a different deployment model (requires careful loader-aware interpretation).

Classification such as “single wallet” versus “multisig” should only be displayed when UseKratose can deterministically identify the controlling account type or through a trusted integration. Do not infer governance from name labels.

17. IDL and Program Metadata

Canonical program metadata and IDLs add semantic context. The Solana program-metadata tooling allows upgrade authorities to publish canonical IDLs and security.txt-like metadata. The IDL explorer resolves canonical metadata and can fall back to legacy Anchor IDLs. [R5]

Useful metadata

- Program name and version
- Public source repository
- Security contacts/policy
- Auditor references
- Canonical IDL
- Description and documentation links

IDL diff targets

- Instructions added or removed
- Instruction argument shape changes
- Account list/signer/writable changes where represented
- Type additions/removals
- Error definitions
- Events and account layouts

An IDL is developer-provided metadata, not proof of runtime behavior. UseKratose must label it as context and keep executable fingerprints authoritative.

18. Verified Builds Integration

Solana verified builds connect public source code to the deployed executable through deterministic builds and hash comparison. The verification metadata can include program address, repository URL, commit hash, and build arguments. A verified build confirms that a given source revision reproduces the on-chain executable; it does not by itself mean the code is secure. [R4]

UseKratose integration model

- Detect whether verification metadata exists for the current deployment.
- Record repository and commit when available.
- If a deployment hash changes but verification metadata still points to an older commit/build, mark verification stale until re-verified.

- Use the source commit transition to power semantic source diffing.
- Never equate “verified” with “audited” or “safe.”

19. Source and Semantic Diffing

The diff engine has layers. Every event should work at Level 1; richer levels activate only when evidence exists.

Level	Inputs	Output
L1 Deployment	ProgramData/account bytes	Hash, size, slot, authority change
L2 Metadata	IDL/program metadata	Instruction/type/metadata changes
L3 Verified source	Repository + old/new commits	Files/functions changed, security-sensitive patterns
L4 Dynamic analysis	Simulation/test harness later	Behavioral changes under selected transactions

Security-sensitive source patterns

Future source analysis can prioritize changes around signer checks, PDA derivations, account ownership constraints, CPI targets, token authority logic, arithmetic, oracle validation, admin/upgrade instructions, and emergency controls. The MVP should not claim complete vulnerability detection from source diffing; it should use these patterns to prioritize human review.

20. Security Event Model

A security event is the user-facing representation of a meaningful transition between two snapshots. It must be reproducible from stored evidence.

```
type SecurityEvent = {
  id: string;
  programId: string;
  fromSnapshotId: string;
  toSnapshotId: string;
  detectedAt: string;
  eventTypes: Array<
    "EXECUTABLE_CHANGED" |
    "AUTHORITY_CHANGED" |
    "BECAME_IMMUTABLE" |
    "IDL_CHANGED" |
    "VERIFICATION_STALE" |
    "METADATA_CHANGED"
  >;
  severity: "info" | "low" | "medium" | "high" | "critical";
  facts: Record<string, unknown>;
  explanation?: string;
};
```

Evidence-first UI

The change detail screen should render raw evidence before narrative. A user must be able to see old/new hash, old/new authority, slots, and metadata state without trusting the prose explanation.

21. Risk Classification

For the MVP, severity should be rule-based rather than a mysterious machine-learning score. A transparent rules engine is easier to test, easier to defend to judges, and safer for users.

Event	Default severity	Reason
Executable hash changed	High	Previous binary no longer matches current deployment
Upgrade authority changed	High	Control over future code changed
Program became immutable	Info/Low	Reduces future upgrade surface; still record transition
Canonical IDL changed only	Medium	Interface assumptions may have changed
Metadata/contact text changed only	Info	Context change, not code proof
Verified-build state stale after executable change	High	Source/deployment relationship must be re-established

Severity must remain configurable later because different organizations have different policies. In enterprise versions, customers should be able to define “any authority change = critical” or “any executable change outside release window = critical.”

22. AI Explanation Layer

AI is an interpreter, not the sensor. The blockchain and deterministic diff engine establish facts; the model turns those facts into a concise explanation, suggests questions for human review, and summarizes source/IDL differences.

Prompt contract

SYSTEM: You explain deterministic security events. Never invent changes.

INPUT:

- before snapshot facts
- after snapshot facts
- deterministic diff
- optional source/IDL diff

OUTPUT:

1. What changed
2. Why it may matter
3. What is unknown
4. Recommended human checks
5. Evidence references

Failure policy

If the AI call fails or is unavailable, the security event must still be complete. The UI shows facts and a template-generated summary. This prevents model availability or hallucination from becoming a security dependency.

23. Backend Architecture

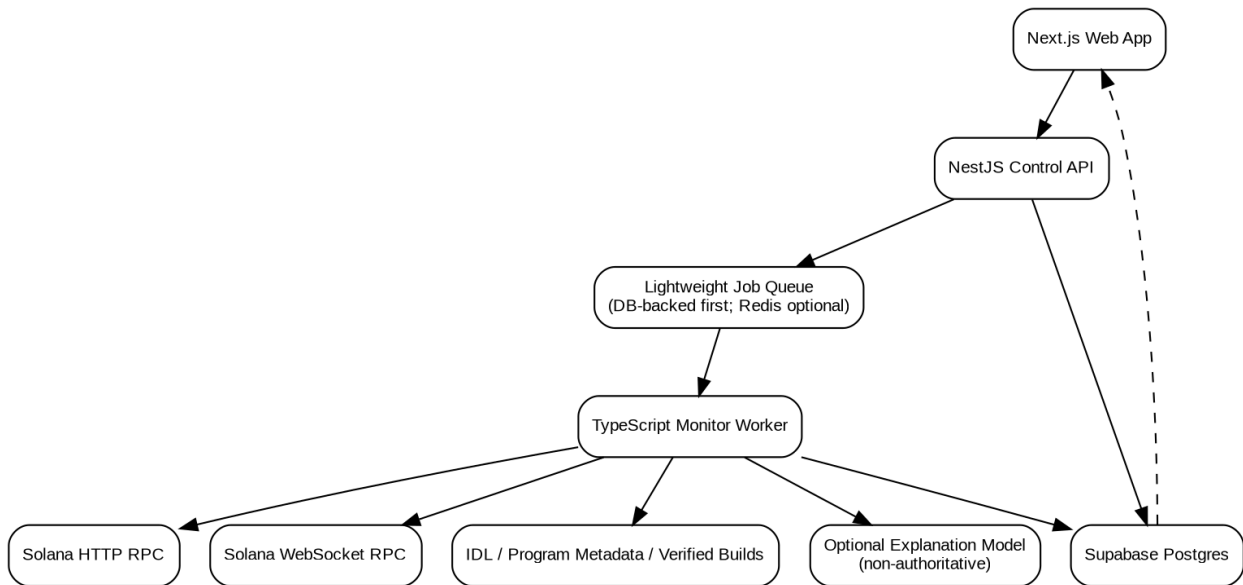


Figure: MVP service architecture.

A pragmatic stack for the hackathon is Next.js for the web UI, NestJS for the control-plane API, a separate TypeScript worker for monitoring, Supabase/Postgres for persistence, and an RPC/WebSocket provider with a free developer tier. Redis is optional at first; a Postgres-backed jobs table can be sufficient for tens or hundreds of monitored programs.

Service boundaries

Service	Responsibility
Web	Authentication UX, project/program views, event diff UI
Control API	CRUD, permissions, API keys, public API
Monitor worker	Subscriptions, reconnection, scheduling, reconciliation

Analyzer	Snapshot normalization, hashes, metadata fetch, diff
Notifier	Webhooks/in-app delivery with retry
Database	Durable state, idempotency, event history

24. Monitoring Worker

The monitor worker owns long-lived network state. It maintains a map from ProgramData addresses to subscriptions, heartbeats WebSocket connections, resubscribes after reconnects, and enqueues reconciliation rather than performing heavy analysis in the socket callback.

Worker state machine

```

UNMONITORED -> RESOLVING -> BASELINING -> MONITORING
MONITORING --pubsub--> RECONCILING -> MONITORING
MONITORING --disconnect--> DEGRADED -> RECONNECTING -> MONITORING
RECONCILING --change--> ANALYZING -> MONITORING
RECONCILING --no-change--> MONITORING

```

Operational rule

Every network callback must be cheap. Hashing large executable payloads, metadata fetching, source diffing, and AI explanation happen in jobs so one slow program cannot block subscription processing for all programs.

25. Frontend Architecture

The frontend is a security operations interface, not a marketing dashboard. Information hierarchy must make status and evidence obvious. Avoid excessive animation, gradient-heavy cards, or fake live metrics. Use clear typography, timeline views, before/after comparisons, and copyable addresses/hashes.

Component map

- ProjectSwitcher
- ProgramStatusCard
- ProgramIdentityPanel
- AuthorityPanel
- CurrentSnapshotPanel
- VersionTimeline
- ChangeEventCard
- DiffTable
- EvidenceDrawer
- AlertDestinationForm
- AddProgramWizard

Data freshness

Every program page should display “last reconciled” and monitoring health. A stale page that looks current is worse than an explicit degraded state.

26. Database Design

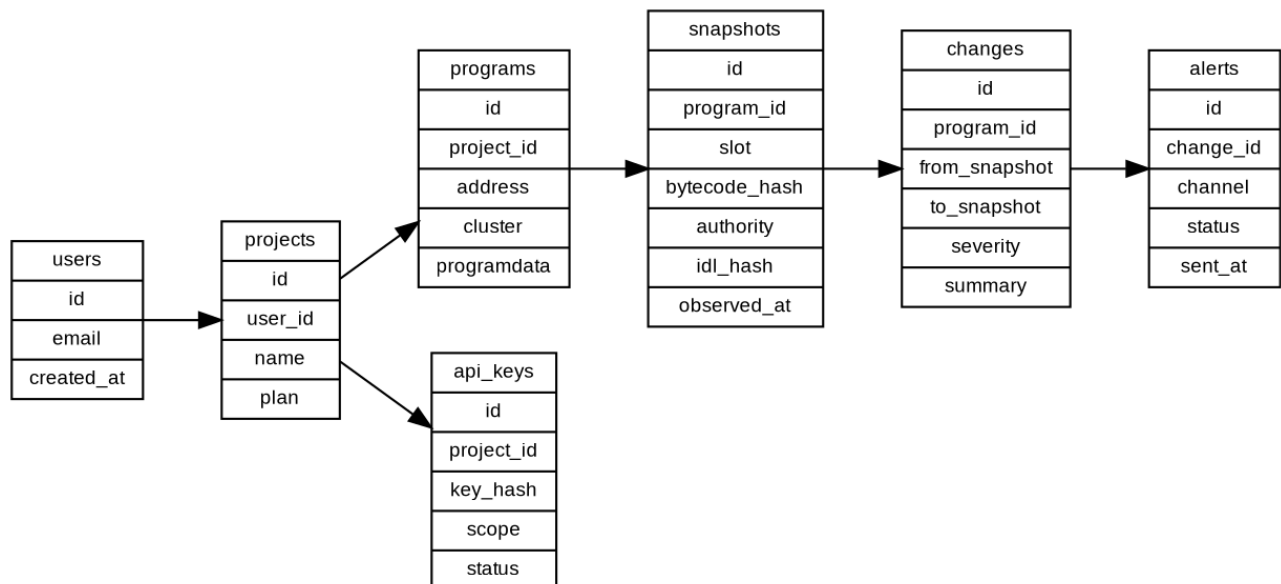


Figure: simplified relational model.

Postgres is the system of record. The schema should preserve history instead of updating a single mutable “current program” row. The current state is derived from the latest

valid snapshot; previous snapshots remain immutable except for backfilled enrichment fields that do not change the original on-chain observation.

Key constraints

- Unique program per project/cluster/address.
- Unique snapshot by program_id + fingerprint.
- Unique event by from_snapshot + to_snapshot.
- API keys stored only as hashes after creation.
- Webhook secrets encrypted at rest if platform support permits.
- Indexes on program_id, observed_at, detected_at, project_id.

27. API Design

The public API begins as read-mostly security state. Write operations for projects and monitors remain authenticated management endpoints.

```
GET /v1/programs/{address}?cluster=mainnet-beta
GET /v1/programs/{address}/snapshots
GET /v1/programs/{address}/events
GET /v1/events/{eventId}
POST /v1/projects/{projectId}/programs
POST /v1/projects/{projectId}/webhooks
POST /v1/projects/{projectId}/api-keys
```

Example response

```
{
  "programId": "...",
  "cluster": "mainnet-beta",
  "monitoring": "healthy",
  "deployment": {
    "hash": "sha256:...",
    "slot": 123456789,
    "upgradeAuthority": "..."
  },
  "verification": {
    "state": "current|stale|unknown"
  },
  "latestEvent": null
}
```

28. Authentication and API Keys

Human users authenticate through a managed auth provider. API clients receive project-scoped keys. Keys are displayed once at creation, stored as hashes, revocable, and optionally scoped to read-only or webhook-management permissions later.

MVP abuse controls

- Rate limit authentication and program-add endpoints.
- Limit number of monitored programs on free accounts.
- Validate program addresses before scheduling monitors.
- Protect public endpoints from unconstrained metadata fetch fan-out.
- Do not accept arbitrary URLs for server-side fetching without an allowlist or safe fetch proxy.

29. Webhooks and Notifications

Webhooks are the most useful first alert channel because they prove UseKratose can integrate into real operational workflows. Email is valuable but not necessary for the first demo if it complicates infrastructure.

Webhook payload

```
{
  "type": "program.security_event",
  "eventId": "evt_...",
  "programId": "...",
  "severity": "high",
  "changes": ["EXECUTABLE_CHANGED", "VERIFICATION_STALE"],
  "detectedAt": "2026-09-20T12:00:00Z",
  "dashboardUrl": "..."
}
```

Delivery rules

Sign webhook payloads, retry failed deliveries with exponential backoff, store response codes, and let users see delivery history. Do not block event creation on notification success.

30. Caching and Rate Limiting

The biggest infrastructure constraint with a zero-budget build is upstream RPC quota. Caching and scheduling matter more than micro-optimizing application code.

- Cache stable metadata such as program owner and ProgramData mapping until a reconciliation invalidates it.
- Deduplicate HTTP reads for the same program when multiple users monitor it.
- Share one account subscription per unique ProgramData address across customers.
- Apply jitter to periodic reconciliations so all programs are not fetched at the same second.
- Back off on provider 429 responses and surface monitoring degradation.

31. Error Handling

Failure	Expected behavior
Invalid address	Reject before creating monitor
Account not executable	Explain that address is not a program
Unsupported loader	Create unsupported status; do not pretend monitoring is active
RPC 429	Retry with backoff; mark degraded if persistent
WebSocket disconnect	Reconnect and run reconciliation
Metadata unavailable	Continue core monitoring; show unavailable
AI unavailable	Use deterministic summary
Database conflict	Use unique constraints/idempotent upsert
Webhook fails	Retry independently; event remains valid

32. Observability

A security product must be observable itself. The team needs to know whether missing events are caused by the chain, RPC provider, subscription health, worker backlog, analyzer failure, or notification failure.

Minimum telemetry

- active subscriptions
- reconnect count
- last message time per socket
- reconciliation duration
- RPC success/error/429 rates
- snapshot creation count
- events created by type
- job queue depth
- oldest pending job age
- webhook success rate
- last successful reconcile per program

The program dashboard should expose a subset of this operational health to users so monitoring status is transparent.

33. Security of UseKratose

UseKratose monitors security-sensitive systems, so compromise of UseKratose could cause false reassurance or false alerts. The MVP should therefore minimize privileges and avoid custody entirely.

Threat model

Threat	Mitigation
Attacker modifies stored snapshot	Append-only history where possible; hashes; audit logs
RPC provider lies/stales	Finalized reads; optional second-provider verification for high-severity events later
Webhook endpoint abused	Signed payloads; allow deletion/rotation
SSRF through metadata URLs	Strict URL validation/allowlist/proxy
API key theft	Hash keys; scopes; rotation
Account takeover	Managed auth, MFA later
AI hallucination	Facts remain authoritative; explanation labeled and reproducible

No private keys

Monitoring a public program should never require the user's upgrade authority or deployment key. If future integrations need signed proof or CI/CD release attestations, use explicit wallet-signing flows rather than collecting secrets.

34. Privacy and Data Handling

Most monitored data is public blockchain information. Private data consists mainly of customer account details, project organization, alert destinations, API keys, and internal notes. Collect the minimum necessary data and separate public program profiles from private project configuration.

- Document what becomes public before a user publishes a profile.
- Never expose webhook URLs or API keys.
- Allow account/project deletion while preserving only legally/operationally necessary anonymous system records.
- Use clear retention rules for raw RPC payloads versus normalized snapshots.

35. Testing Strategy

Testing must focus on correctness of change detection and idempotency before visual polish.

Test layers

Layer	Examples
Unit	loader parser, normalization, hash generation, diff rules, severity rules
Fixture	known Program/ProgramData account bytes, metadata JSON variants
Integration	devnet RPC reads, WebSocket reconnect, database constraints
End-to-end	add program -> baseline -> deploy v2 -> detect -> event -> webhook
Chaos	disconnect socket, duplicate notifications, 429 responses, worker restart
UI	before/after diff readability, stale/degraded states

Golden fixtures

Save anonymized or test-program account payloads for v1 and v2. The same fixtures should always generate the same snapshots and event output. This gives the team confidence when parser code changes.

36. Test Fixtures and Demo Program

Build a tiny devnet program specifically to demonstrate lifecycle monitoring. Version 1 should be deliberately simple. Version 2 should introduce a visible, non-dangerous semantic change such as a new admin-config instruction or a changed account schema. The goal is to prove detection, not to demonstrate an exploit.

Demo sequence

18. Deploy v1 and record baseline.
19. Publish optional IDL/metadata for v1.
20. Open UseKratose program page on screen.
21. Deploy v2 to the same Program ID.
22. Wait for finalized notification/reconciliation.
23. Show automatic event creation.
24. Open diff: new hash, slot, IDL change, verification stale.
25. Show webhook payload or external notification.

37. Deployment and Environments

Environment	Purpose	Cluster
Local	Fast development with fixtures and optional local validator	local/devnet
Preview	Per-branch UI/API validation	devnet
Staging	End-to-end worker and demo rehearsals	devnet
Production hackathon	Real users and mainnet monitoring	mainnet-beta + devnet

Keep RPC URLs, WebSocket URLs, auth secrets, database credentials, webhook signing secret, and AI provider keys in environment variables. Never commit them. Separate the monitoring worker from serverless request handlers if the platform does not support persistent sockets.

38. Cost-Minimized Infrastructure

The MVP can be built without a validator because UseKratose only needs standard account reads/subscriptions and public metadata. The zero-budget strategy is to minimize the number of unique upstream reads and keep the monitored set intentional.

Component	Low-cost choice	Why
Web/API	Free-tier frontend/serverless or small container	Low request volume
Database/Auth	Supabase free tier or equivalent	Postgres + auth
Monitor worker	Free/credit container host or local demo worker initially	Persistent WebSocket needed
RPC	Developer/free tier; devnet public for development	No validator required
Queue	Postgres jobs first	Avoid extra Redis service
AI	Optional free credits / disabled fallback	Not core correctness dependency

The first scaling expense should be reliable RPC capacity, not a custom validator. Owning nodes only becomes relevant when customer volume or reliability economics justify it.

39. Performance and Scaling

UseKratose's scaling unit is the number of unique monitored programs, not the number of customer accounts. If 100 customers monitor the same popular program, the system should keep one upstream monitor and fan out the resulting event to all subscribed projects.

Scaling path

- Phase 1: one worker process, in-memory subscription registry, Postgres jobs.
- Phase 2: partition monitored programs across workers using leases.
- Phase 3: dedicated queue and distributed subscription ownership.
- Phase 4: multi-provider RPC routing and regional workers.
- Phase 5: archival/event data warehouse for large-scale analytics.

40. Product Analytics

Analytics must answer whether the product is becoming operationally useful, not merely whether people visit the landing page.

Metric	Meaning
Monitored programs	Installed base
Weekly active security teams	Real operational use
Unique change events detected	Product doing work
Programs with >30 days monitoring	Retention
Webhook/API integrations	Workflow embed
Claimed/public profiles	Distribution conversion
Time-to-detect	Operational performance
Event review rate	Usefulness of alerts
Free-to-paid conversion	Business validation

41. Pricing and Packaging

Pricing should be validated with users rather than treated as final. A sensible packaging unit is monitored programs plus retention/integration capability.

Plan	Illustrative price	Target	Limits/features
Free	\$0	Developers/hackath on teams	1 program, basic history, in-app events
Builder	\$29–49/mo	Small teams	3–5 programs, webhooks, longer history

Protocol	\$149–299/mo	Live protocols	10–25 programs, API, policies, team access
Security	\$499–999+/mo	Auditors/security teams	Portfolio monitoring, client workspaces, reports
Enterprise	Custom	Wallets/exchanges/institutions	Bulk API, SLA, retention, custom rules

These figures are starting hypotheses. During the hackathon, the stronger validation is whether users ask to keep monitoring after the event, whether they integrate a webhook, and whether at least one team discusses paying.

42. Go-to-Market

The initial go-to-market is founder-led and ecosystem-specific. Do not buy ads. Acquire users by producing a useful security artifact for a program they already care about.

Primary motion

26. Identify active Solana teams and hackathon builders.
27. Offer free monitoring during the hackathon.
28. Generate a security/deployment profile from public facts.
29. Ask the team to claim the program and connect a webhook.
30. Interview them after the first real release/change event.
31. Convert the highest-need teams to paid plans after the hackathon.

Message

“Audits are snapshots. UseKratose keeps watching the program after the audit and tells you when the deployment or control assumptions change.” This is clearer than leading with AI, hashes, or loader internals.

43. Distribution Loops

Public program profile loop

A public profile can show objective deployment facts and monitoring history. Teams can claim profiles, add official metadata, and enable monitoring. Search traffic and social sharing create inbound acquisition.

Auditor loop

Auditors can monitor previously audited clients. When a deployment changes, UseKratose creates a re-review trigger, turning the product into a retention and repeat-engagement tool rather than an auditor replacement.

API loop

Wallets, explorers, and aggregators can display deployment-change context. Every integration increases the usefulness of UseKratose data and creates new monitored-program demand.

44. Competitive Positioning

UseKratose should not claim that nobody monitors upgrades. Existing security and operations tools can alert on Solana account/program changes, and verified-build systems can periodically re-verify source/deployment relationships. The differentiation must be the complete lifecycle: baseline, continuous monitoring, deployment fingerprint, authority/metadata/verification diff, historical evidence, and an operational security event that explains what previous trust assumptions became stale.

Alternative	What it does well	UseKratose wedge
Generic RPC/account monitor	Detects account changes	Security-specific normalization, history, diff, trust-state model
Verified builds	Proves source matches deployment	Continuous lifecycle monitoring and change events around that proof
One-time audit	Deep expert review at a point in time	Detects when the reviewed state is no longer current

Block explorer	Shows chain data	Operational alerts, policy, history, API for teams
AI code scanner	Finds code issues	Deployment reality and deterministic change evidence first

45. Sales Playbook

First outreach

Keep outreach short and specific. Offer a concrete free action: monitor one production/devnet program and notify the team on the next change. Ask for the public Program ID, not access to secrets.

Discovery questions

- How do you currently know that production matches the release you intended?
- Who gets notified when the program is upgraded?
- How do you track upgrade authority changes?
- After an audit, how do you communicate later upgrades to the auditor/integrators?
- Would a webhook on deployment drift fit your release/security workflow?
- Which change would be severe enough to wake someone up?

Conversion trigger

The best conversion moment is after UseKratose catches a real upgrade and produces a useful event. The team has then experienced the product's value rather than buying a hypothetical dashboard.

46. Customer Discovery

During the hackathon, customer discovery is part of product development. Maintain a structured log of interviews, program addresses monitored, requested features, objections, and whether the team would continue after the hackathon.

Hypothesis	Evidence needed
Teams care about post-audit drift	Multiple teams describe current process as manual/absent
Authority changes matter	Security teams request alerts or policy
Semantic diff is valuable	Users open/review events instead of only acknowledging alerts
API is valuable	At least one external integration prototype
Recurring pricing works	Teams discuss monthly monitoring rather than one-time report

47. Colosseum Strategy

Colosseum describes its hackathons as startup competitions and evaluates founder-market fit, insight, product execution, market size, communication, viability, and traction. The submission should therefore tell a company story, not merely a technical demo. [R6]

What judges should understand in 30 seconds

- Problem: an audit is a snapshot, while upgradeable programs continue changing.
- Product: UseKratose continuously verifies the deployed program and detects security-relevant drift.
- Proof: live Solana upgrade detection and deterministic diff.
- Market: protocols, auditors, wallets, exchanges, integrators.
- Business: recurring monitoring SaaS + security API.
- Traction: real programs/teams monitored during the hackathon.

Repository strategy

Keep commit history active throughout the hackathon, document architecture, include a reproducible demo/test program, and prioritize the core monitoring loop. Colosseum explicitly says it looks for significant work during the hackathon and strategic prioritization rather than any particular framework. [R6]

48. Demo Script

Target a 2–3 minute product video. The live upgrade is the centerpiece.

Time	Scene	Message
0:00–0:20	Problem	“This program was audited. But what happens after it is upgraded?”
0:20–0:45	Baseline	Show monitored devnet program: hash, authority, current verification status
0:45–1:20	Real upgrade	Deploy v2 from terminal; no manual dashboard edits
1:20–1:50	Detection	Dashboard updates: new slot/hash, event created
1:50–2:20	Diff	Show IDL/source/authority differences and verification stale state
2:20–2:40	Integration	Show webhook event
2:40–3:00	Business/traction	Programs monitored, target customer, paid monitoring/API model

49. Four-Week Hackathon Roadmap

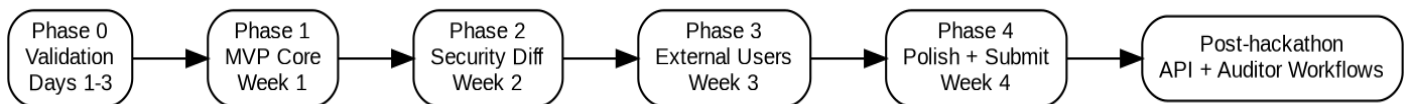


Figure: hackathon and post-hackathon roadmap.

Week 1: Make the chain loop undeniable

- Create repo/workspace and deployment environments.
- Implement program resolver.
- Implement ProgramData parser for supported loader.
- Implement baseline snapshot and hashes.
- Implement WebSocket watcher.
- Implement finalized HTTP reconciliation.
- Deploy demo program v1/v2 and prove detection end-to-end.

Week 2: Turn changes into security information

- Authority diff.
- Version timeline.
- IDL/program metadata fetch.
- Deterministic diff engine.
- Rule-based event severity.
- Webhook delivery.
- Polish program/change screens.

Week 3: External reality

- Onboard real Solana teams.
- Monitor real programs.
- Fix false positives and unsupported cases.
- Add verified-build/source context.
- Add explanation layer with deterministic fallback.
- Publish clear docs/API examples.

Week 4: Submission quality

- Stabilize worker/reconnect behavior.
- Stress test duplicates and provider failures.
- Collect traction metrics and testimonials/quotes with permission.
- Polish onboarding.
- Record demo repeatedly.
- Finalize README, architecture, security limitations, business model, and submission.

50. Post-Hackathon Roadmap

Horizon	Product
0–3 months	Reliable mainnet monitoring, team workspaces, webhooks, API, public profiles, verified-build integration
3–6 months	Auditor portfolios, policy rules, Slack/Discord, release/CI integration, multi-provider validation

6–12 months	Wallet/explorer API integrations, dependency graph, source diff intelligence, enterprise retention/reporting
12–24 months	Cross-program dependency risk, transaction-behavior monitoring, cross-chain expansion where deployment models support comparable evidence

Product expansion principle

Every expansion must preserve the core identity: UseKratose is the evidence and monitoring layer that tells systems whether the on-chain software they trust has changed. Features that do not reinforce that identity should be treated skeptically.

51. KPI Framework

Stage	North-star supporting metrics
Hackathon	Programs monitored, external teams, real events detected, webhook integrations
Early SaaS	30-day retained programs, weekly active teams, events reviewed, paid conversion
API growth	Monthly API consumers, request volume, integrated wallets/explorers
Security network	Auditor portfolios, re-review triggers, monitored TVL/integration footprint (reported carefully)

Avoid vanity metrics such as page views unless they lead to program claims or monitors. A security product wins by becoming part of workflows.

52. Risk Register

Risk	Impact	Mitigation
Existing tools cover enough of the problem	High	Differentiate on trust lifecycle + semantic security diff + workflow
Free RPC unreliable	High	Dual detection/reconciliation; provider abstraction later
Unsupported loader/program format	Medium	Clear compatibility matrix; modular resolver
IDL/source unavailable	Medium	Core binary/authority monitoring still works
False security implication	High	Evidence-first language; no “safe” guarantee
AI hallucination	High	AI non-authoritative; deterministic facts always shown
No user traction	High	Start outreach in week 1, not after product completion
Scope explosion	High	Freeze P0; defer marketplace/cross-chain/formal verification
Worker cannot run on chosen serverless host	Medium	Deploy persistent worker separately
Brand confusion	Low/Medium	Use consistent UseKratose identity and security positioning

53. Team Operating Model

For a solo or very small hackathon team, process must reduce context switching. Maintain one backlog, one architecture decision log, and one daily definition of “demo still works.”

Daily rhythm

- Morning: choose one measurable product outcome.
- Build: work in small commits tied to an issue.
- Midday: run end-to-end demo path.
- Evening: update changelog/Colosseum progress and note blockers.
- Every 2–3 days: talk to at least one external builder/security person.

Decision rule

When choosing between a new feature and making the live upgrade demo more reliable, choose reliability until the core loop is boringly repeatable.

54. Definition of Done

The hackathon MVP is done only when the following can be demonstrated from a fresh account and a fresh deployment without editing the database manually.

- User can sign in and add a valid program.
- System resolves the correct tracked deployment account for supported programs.
- Baseline snapshot is stored and rendered.
- Worker detects a real upgrade.
- HTTP reconciliation confirms the finalized new state.
- New snapshot has a different deterministic fingerprint.
- Authority/IDL/verification differences are shown when available.
- Exactly one event is created for the transition.
- User receives an alert.
- Dashboard shows monitoring health and last reconciliation.
- Demo survives a worker restart and a duplicate notification.
- README documents limitations and architecture.

55. Backlog and Future Modules

Module	Why it belongs later
Auditor marketplace	Needs supply/demand; core monitoring first

Formal verification	Specialized research effort; not required to prove wedge
Wallet extension	Distribution surface after API is trusted
Dependency graph	Powerful once enough programs are monitored
Cross-chain	Dilutes Solana execution during hackathon
Own RPC/validator	Capital-intensive and unnecessary at early scale
Token/governance	Does not solve core customer problem
Behavioral transaction monitoring	Valuable but distinct data plane; add after deployment monitoring works
Release attestations	Strong enterprise/CI feature after basic verification lifecycle

Appendix A: Detailed MVP Backlog

Epic	Definition
EPIC-01 Program onboarding	Validate address; cluster; resolve account; error states; optional display name.
EPIC-02 Baseline	Fetch finalized state; normalize; hash; persist; display.
EPIC-03 Realtime watcher	Subscribe; heartbeat; reconnect; shared subscriptions.
EPIC-04 Reconciliation	Scheduler; idempotency; 429 handling; jitter.
EPIC-05 Diff engine	Hash, size, authority, metadata, IDL, verification state.
EPIC-06 Events	Rules; severity; facts; timeline.
EPIC-07 Alerts	Webhook signing; retries; delivery log.
EPIC-08 UX	Dashboard; program detail; diff detail; degraded states.
EPIC-09 Demo	v1/v2 program; repeatable deploy script; seeded metadata.
EPIC-10 Docs	README; API examples; architecture; known limitations.

EPIC-01 Program onboarding

Validate address; cluster; resolve account; error states; optional display name.

- Acceptance criteria are observable from the UI or API.
- Failure states are tested, not only the happy path.
- The epic must not introduce a hidden dependency on private keys or paid infrastructure.
- Every database write is idempotent or protected by a uniqueness constraint.

EPIC-02 Baseline

Fetch finalized state; normalize; hash; persist; display.

- Acceptance criteria are observable from the UI or API.
- Failure states are tested, not only the happy path.
- The epic must not introduce a hidden dependency on private keys or paid infrastructure.
- Every database write is idempotent or protected by a uniqueness constraint.

EPIC-03 Realtime watcher

Subscribe; heartbeat; reconnect; shared subscriptions.

- Acceptance criteria are observable from the UI or API.
- Failure states are tested, not only the happy path.
- The epic must not introduce a hidden dependency on private keys or paid infrastructure.
- Every database write is idempotent or protected by a uniqueness constraint.

EPIC-04 Reconciliation

Scheduler; idempotency; 429 handling; jitter.

- Acceptance criteria are observable from the UI or API.
- Failure states are tested, not only the happy path.
- The epic must not introduce a hidden dependency on private keys or paid infrastructure.
- Every database write is idempotent or protected by a uniqueness constraint.

EPIC-05 Diff engine

Hash, size, authority, metadata, IDL, verification state.

- Acceptance criteria are observable from the UI or API.
- Failure states are tested, not only the happy path.

- The epic must not introduce a hidden dependency on private keys or paid infrastructure.
- Every database write is idempotent or protected by a uniqueness constraint.

EPIC-06 Events

Rules; severity; facts; timeline.

- Acceptance criteria are observable from the UI or API.
- Failure states are tested, not only the happy path.
- The epic must not introduce a hidden dependency on private keys or paid infrastructure.
- Every database write is idempotent or protected by a uniqueness constraint.

EPIC-07 Alerts

Webhook signing; retries; delivery log.

- Acceptance criteria are observable from the UI or API.
- Failure states are tested, not only the happy path.
- The epic must not introduce a hidden dependency on private keys or paid infrastructure.
- Every database write is idempotent or protected by a uniqueness constraint.

EPIC-08 UX

Dashboard; program detail; diff detail; degraded states.

- Acceptance criteria are observable from the UI or API.
- Failure states are tested, not only the happy path.
- The epic must not introduce a hidden dependency on private keys or paid infrastructure.
- Every database write is idempotent or protected by a uniqueness constraint.

EPIC-09 Demo

v1/v2 program; repeatable deploy script; seeded metadata.

- Acceptance criteria are observable from the UI or API.
- Failure states are tested, not only the happy path.
- The epic must not introduce a hidden dependency on private keys or paid infrastructure.
- Every database write is idempotent or protected by a uniqueness constraint.

EPIC-10 Docs

README; API examples; architecture; known limitations.

- Acceptance criteria are observable from the UI or API.
- Failure states are tested, not only the happy path.
- The epic must not introduce a hidden dependency on private keys or paid infrastructure.
- Every database write is idempotent or protected by a uniqueness constraint.

Appendix B: Suggested Repository Structure

```

usekratose/
  apps/
    web/           # Next.js UI
    api/           # NestJS control API
    monitor-worker/ # persistent subscriptions + reconciliation
  packages/
    solana-monitor/ # loader parsing, RPC adapters, snapshots
    diff-engine/    # deterministic change analysis
    shared-types/   # DTOs and event schemas
    ui/             # reusable components
  programs/
    demo-program/   # devnet v1/v2 hackathon fixture
  infra/
    migrations/
    docker/
  docs/
    architecture.md
    threat-model.md
    api.md
  scripts/
    deploy-demo-v1.sh
    deploy-demo-v2.sh
  .github/
    workflows/
  README.md

```

Keep the Solana parsing/diff logic in packages that can be tested independently from HTTP or React. The demo program belongs in the same repository or a clearly linked companion repository so judges can reproduce the lifecycle demonstration.

Appendix C: Database Schema Sketch

```

users(id, email, created_at)
projects(id, user_id, name, plan, created_at)
programs(id, project_id, address, cluster, loader, programdata_address,
  monitoring_status, last_reconciled_at, created_at)
snapshots(id, program_id, fingerprint, slot, executable_hash,
  account_data_hash, upgrade_authority, idl_hash, metadata_hash,
  verified_build_state, raw_refs_json, observed_at)
security_events(id, program_id, from_snapshot_id, to_snapshot_id,
  severity, types_json, facts_json, explanation,
  detected_at, reviewed_at)
webhooks(id, project_id, url_ciphertext, secret_ciphertext, active)
webhook_deliveries(id, webhook_id, event_id, attempt, status_code,
  next_retry_at, delivered_at)
api_keys(id, project_id, prefix, key_hash, scopes_json, revoked_at, created_at)
jobs(id, type, dedupe_key, payload_json, status, attempts,
  run_after, locked_at, created_at)

```

Appendix D: REST API Contract Details

All timestamps are ISO-8601 UTC. Addresses remain base58 strings. API errors have a stable code and human-readable message. Public read endpoints may be rate-limited independently from authenticated project endpoints.

Endpoint	Auth	Notes
GET /v1/programs/:address	API key optional/public later	Current normalized state
GET /v1/programs/:address/snapshots	API key	Paginated history
GET /v1/programs/:address/events	API key	Filter by severity/type
POST /v1/projects/:id/programs	Session/API key	Add monitor
DELETE /v1/projects/:id/programs/:programId	Session/API key	Stop project subscription; shared upstream monitor may remain
POST /v1/projects/:id/webhooks	Session	Create alert destination
POST /v1/projects/:id/api-keys	Session	Create one-time visible key

Appendix E: Event Examples

Executable change

```
{
  "eventTypes": ["EXECUTABLE_CHANGED", "VERIFICATION_STALE"],
  "severity": "high",
  "facts": {
    "oldHash": "sha256:aaa...",
    "newHash": "sha256:bbb...",
    "oldSlot": 100,
    "newSlot": 205,
    "upgradeAuthorityChanged": false
  }
}
```

Authority change only

```
{
  "eventTypes": ["AUTHORITY_CHANGED"],
  "severity": "high",
  "facts": {
    "oldAuthority": "Old...",
    "newAuthority": "New...",
    "executableChanged": false
  }
}
```

Became immutable

```
{
  "eventTypes": ["BECAME_IMMUTABLE"],
  "severity": "info",
  "facts": {
    "oldAuthority": "Auth...",
    "newAuthority": null
  }
}
```

Appendix F: Monitoring Pseudocode

```
async function baseline(program) {
  const resolved = await resolveProgram(program.address, program.cluster);
  const state = await fetchFinalizedState(resolved);
  const enriched = await enrichBestEffort(state);
  const snapshot = createSnapshot(enriched);
  await insertSnapshotIfAbsent(snapshot);
  await ensureSubscription(resolved.programDataAddress ?? resolved.programId);
}
```

```

async function reconcile(programId, reason) {
  const program = await db.program.find(programId);
  const state = await fetchFinalizedState(program);
  const next = createSnapshot(await enrichBestEffort(state));
  const prev = await db.snapshot.latest(programId);
  if (prev && prev.fingerprint === next.fingerprint) return;
  await db.transaction(async tx => {
    const saved = await tx.snapshot.insertUnique(next);
    const event = diffSnapshots(prev, saved);
    await tx.securityEvent.insertUnique(event);
    await tx.job.enqueueNotifications(event.id);
  });
}

```

Appendix G: Diff Rules Specification

Rule ID	Condition	Event
D-001	old.executable_hash != new.executable_hash	EXECUTABLE_CHANGED
D-002	old.upgrade_authority != new.upgrade_authority	AUTHORITY_CHANGED
D-003	old.upgrade_authority != null AND new == null	BECAME_IMMUTABLE
D-004	old.idl_hash != new.idl_hash	IDL_CHANGED
D-005	executable changed AND old verification no longer matches current	VERIFICATION_STALE
D-006	old.metadata_hash != new.metadata_hash	METADATA_CHANGED

Rules must be deterministic and versioned. Store the rule-engine version on each event so historical events can be reproduced even after classification logic evolves.

Appendix H: UX Copy Guide

Avoid	Prefer
“This contract is safe.”	“No monitored deployment change detected since [time].”
“AI says risk 92/100.”	“Executable changed; previous verification no longer matches current deployment.”
“Hacked / dangerous.”	“Review recommended: upgrade authority changed.”
“Verified = secure.”	“Verified build: source reproduces deployed executable; security review is separate.”
“Real-time” when degraded	“Monitoring degraded; last successful reconcile [time].”

Security language must be precise because false reassurance damages trust. UseKratose should earn credibility by stating what it knows, what it does not know, and the evidence behind each conclusion.

Appendix I: Threat Model Detail

T1 Upstream data manipulation

A compromised or faulty RPC provider returns incorrect account state.

- Mitigation must be testable or operationally observable.
- Residual risk should be documented rather than hidden.

T2 Missed event

WebSocket notification is lost during disconnect.

- Mitigation must be testable or operationally observable.
- Residual risk should be documented rather than hidden.

T3 Duplicate event

Reconnect or provider replay causes repeated notification.

- Mitigation must be testable or operationally observable.
- Residual risk should be documented rather than hidden.

T4 Metadata spoofing

Non-canonical or malicious metadata is presented as official.

- Mitigation must be testable or operationally observable.
- Residual risk should be documented rather than hidden.

T5 Source mismatch

Repository/commit reference does not reproduce current executable.

- Mitigation must be testable or operationally observable.
- Residual risk should be documented rather than hidden.

T6 User-account compromise

Attacker changes webhook destinations or hides monitors.

- Mitigation must be testable or operationally observable.
- Residual risk should be documented rather than hidden.

T7 Model hallucination

Explanation invents a vulnerability or change.

- Mitigation must be testable or operationally observable.
- Residual risk should be documented rather than hidden.

T8 SSRF

Metadata URL causes backend to request an internal address.

- Mitigation must be testable or operationally observable.
- Residual risk should be documented rather than hidden.

T9 Alert fatigue

Low-value metadata changes overwhelm users.

- Mitigation must be testable or operationally observable.
- Residual risk should be documented rather than hidden.

T10 Product overclaim

Marketing implies safety guarantee beyond evidence.

- Mitigation must be testable or operationally observable.
- Residual risk should be documented rather than hidden.

Appendix J: Customer Interview Script

32. Tell me about the last time you upgraded a Solana program.
33. How did you verify production matched the intended release?
34. Who knew the upgrade happened and how were they notified?
35. Do you publish verified builds or canonical IDLs?
36. How do you handle audits after subsequent releases?
37. Do external integrators ask you which deployed version they are interacting with?
38. Which authority or deployment change would be considered an incident?
39. Would you connect a webhook that fires on a program change?
40. Would you pay monthly to monitor your own programs, dependencies, or audit clients? Why/why not?

Do not pitch after every question. Capture the current workflow and pain first. A strong signal is an existing manual workaround, recurring release checklist, or explicit request to monitor additional programs.

Appendix K: Colosseum Weekly Update Template

Week X Update

What shipped

- ...
- ...

What changed from user feedback

- ...

Product evidence

- Programs monitored: N
- External teams: N
- Security events detected: N
- Integrations/webhooks: N

Demo link

- ...

What we learned

- ...

Next week

- ...

Appendix L: Submission Narrative

Problem

Audits and verified builds describe a specific version. Upgradeable programs continue changing. Integrators and users need a continuous way to know whether the security state they previously trusted still maps to what is deployed.

Insight

The stable Program ID is not enough. Solana already exposes the primitives needed to continuously observe deployment state ProgramData, upgrade authority, account subscriptions, canonical metadata, and verified builds. The missing product is the lifecycle layer that joins them into operational security evidence.

Solution

UseKratose baselines a program, continuously monitors deployment/control state, fingerprints every version, diffs meaningful changes, and alerts teams when previous verification or audit assumptions become stale.

Why this can be a company

Protocols pay for monitoring and workflow integrations; auditors monitor client portfolios; wallets/explorers/integrators consume a machine-readable security-status API. The same trust-lifecycle primitive can later extend to other chains where deployed code, upgrade authority, and source verification can be independently observed.

Appendix M: Acceptance Test Matrix

ID	Scenario	Expected
AT-001	Valid upgradeable devnet program	Baseline created and subscription healthy
AT-002	Invalid base58 input	Rejected with validation error
AT-003	Non-executable account	Rejected as not a program
AT-004	Duplicate add in same project	No duplicate monitor
AT-005	WebSocket event, no finalized change	No security event
AT-006	Executable changes	New snapshot + high-severity event
AT-007	Authority changes only	Authority event with old/new values
AT-008	Program becomes immutable	Immutable transition recorded
AT-009	Socket disconnect	Status degraded then reconnect/reconcile
AT-010	Duplicate notification	One snapshot/event only
AT-011	IDL unavailable	Core event still created
AT-012	AI unavailable	Deterministic summary shown
AT-013	Webhook returns 500	Retry scheduled, event remains visible

Appendix N: Engineering Decision Log (Initial)

Decision	Rationale	Revisit when
Solana-only MVP	Maximum depth and credible hackathon execution	After product-market signal
Loader-v3 first	Clear ProgramData model and common upgrade path	Users need additional loaders
PubSub + HTTP reconciliation	Speed plus completeness	Provider architecture changes
Postgres as system of record	History, constraints, simple operations	Extreme event volume
AI non-authoritative	Security correctness cannot depend on model output	Never for core facts
No own validator	Cost and complexity unnecessary	RPC spend/reliability justifies it

No token	No product need	Only if future protocol economics require it
----------	-----------------	--

Appendix O: Security Review Checklist for Each Release

- No private keys or wallet secrets stored.
- All API keys hashed and revocable.
- Webhook secrets rotated/tested.
- Program address validation tested.
- SSRF protection on remote metadata/source fetches.
- RPC response parsing handles malformed/unexpected data.
- Unique constraints enforce event idempotency.
- Monitoring health visible.
- AI output cannot overwrite deterministic facts.
- Public profile exposes no private project data.
- Error logs do not contain secrets.
- Dependency audit and lockfile committed.

Appendix P: Product Naming and Brand Positioning

UseKratose should be presented as a proper noun and infrastructure/security brand.

Avoid explaining the name in the primary pitch; explain the problem first. The product descriptor does the work: “continuous security verification for Solana programs.”

Layer	Suggested copy
Category	Continuous protocol security
Short descriptor	Continuous security verification for Solana programs
Core line	Audits are snapshots. UseKratose keeps watching after deployment.

Developer line	Detect program upgrades, authority changes, and stale verification automatically.
API line	Machine-readable deployment trust state for Solana programs.

Appendix Q: Open Technical Questions

- Which loader versions will be explicitly supported at launch?
- Which RPC provider offers the most reliable free WebSocket allowance for hackathon volume?
- How should executable bytes be normalized before hashing for each loader?
- What verified-build API/state is stable enough to integrate during the hackathon?
- What canonical IDL sources should be queried and in what precedence order?
- Can the worker host maintain persistent WebSockets on the selected free tier?
- Do we need a second RPC confirmation path for high-severity events before submission?
- What public program examples are safe and useful to include in demo/profile pages?

Resolve these questions with small experiments early. Do not let them remain architecture debates while implementation stalls.

Appendix R: Reference Architecture Sequence

User -> Web: Add Program ID
Web -> API: POST /projects/:id/programs
API -> Resolver: resolve(programId)
Resolver -> Solana HTTP: getAccountInfo(programId)
Resolver -> Solana HTTP: getAccountInfo(programData)
Resolver -> Analyzer: normalize + fingerprint
Analyzer -> DB: insert baseline snapshot
API -> Worker: ensure monitor
Worker -> Solana WS: accountSubscribe(programData)
Solana WS -> Worker: accountNotification
Worker -> Queue: reconcile(programId)
Queue -> Solana HTTP: fetch finalized state
Queue -> Analyzer: create next snapshot
Analyzer -> Diff: compare old/new
Diff -> DB: insert snapshot + security event
DB -> Notifier: delivery job
Notifier -> Customer Webhook: signed event

Web -> API: read latest event

API -> Web: facts + explanation

Appendix S: Glossary

Term	Definition
Program ID	The stable on-chain address applications use to invoke a Solana program.
ProgramData	For upgradeable loader-v3 programs, the account containing deployed executable bytes and upgrade metadata.
Upgrade authority	The authority permitted to upgrade or close an upgradeable program; null means immutable under that model.
Snapshot	UseKratose's normalized evidence bundle for an observed program state.
Fingerprint	Deterministic hash over selected normalized state used to identify a deployment state.
Security event	A structured record describing a meaningful transition between snapshots.
IDL	Interface Description Language describing instructions/accounts/types; useful semantic metadata, not runtime proof.
Verified build	Evidence that a public source revision can reproducibly build to the on-chain executable hash.
Reconciliation	Periodic authoritative fetch used to recover from missed real-time notifications.
Drift	A difference between the current deployed/control state and a previously trusted baseline.
Control plane	User/projects/policies configuration services.
Monitoring plane	Long-lived subscriptions, reconciliation, snapshot and event processing.

Appendix T: References and Source Notes

[R1] Solana Documentation - Programs — Programs are executable sBPF accounts; loader-v3 programs can be upgradeable and become immutable by revoking authority. solana.com/docs/core/programs

[R2] Solana Documentation - Deploying Programs – Describes Program ID, Program account, ProgramData, program authority, deployment/upgrade commands and metadata. solana.com/docs/programs/deploying

[R3] Solana RPC - accountSubscribe – Documents account-change subscriptions and accountNotification payloads. solana.com/docs/rpc/websocket/accountssubscribe

[R4] Solana Documentation / Solana Verified Builds – Explains deterministic source-to-deployment verification by comparing hashes and storing verification metadata. Solana Foundation verified-build documentation/repository.

[R5] Solana Program Metadata / IDL Explorer – Documents canonical IDL and security metadata written by upgrade authority, plus legacy Anchor fallback. idl.solana.com/docs and solana-program.com/program-metadata.

[R6] Colosseum Hackathon – Judging factors include founder-market fit, insight, product/execution, market size, communication, viability, traction, and strategic prioritization in the repository. colosseum.com/hackathon.

Reference notes are included to anchor the technical and competition assumptions used in this specification. Before production launch, refresh dependencies and implementation details against current Solana/Agave documentation and any provider-specific API terms.

Immediate Next Actions: Start Here

This documentation is intentionally comprehensive. The build should start with the smallest experiment that proves the product thesis, not with account management, marketing pages, or AI.

41. Create the repository and demo Solana program.
42. Deploy v1 to devnet and capture Program ID + ProgramData with the Solana CLI.
43. Write a TypeScript script that resolves ProgramData and prints the current executable hash/authority.

44. Add accountSubscribe on ProgramData and log notifications.
45. Deploy v2 to the same Program ID and prove the script detects a real change.
46. Wrap the script in a persistent worker and store snapshots in Postgres.
47. Build the program detail + version timeline UI.
48. Add deterministic diff and webhook.
49. Only after the loop is reliable, add IDL/verified-build enrichment and AI explanation.
50. Start onboarding external Solana teams before the product feels “finished.”

The project is strongest when UseKratose can prove, live and from public blockchain evidence, that it noticed a real program lifecycle change that a user would otherwise have had to track manually.